

CORE CONCEPTS

Persist sessions to external storage

Mirror session transcripts to S3, Redis, or your own backend so any host can resume them.

By default, the SDK writes session transcripts to JSONL files under `~/.claude/projects/` on the local filesystem. A `SessionStore` adapter lets you mirror those transcripts to your own backend, such as S3, Redis, or a database, so a session created on one host can be resumed on another.

Common reasons to use a session store:

- **Multi-host deployments.** Serverless functions, autoscaled workers, and CI runners don't share a filesystem. A shared store lets any replica resume any session.
- **Durability.** Local containers are ephemeral. A store backed by S3 or a database survives restarts and redeploys.
- **Compliance and audit.** Keep transcripts in storage you already govern, with your own retention rules, encryption, and access controls.

The `SessionStore` interface

A `SessionStore` is an object with two required methods, `append` and `load`, and three optional methods. The SDK calls `append` to write transcript entries during a query and `load` to read them back for resume.

```
// Exported from @anthropic-ai/claude-agent-sdk as
// SessionStore, SessionKey, SessionStoreEntry.

type SessionKey = {
  projectKey: string;
  sessionId: string;
  subpath?: string;
};

type SessionStore = {
  // Required
  append(key: SessionKey, entries: SessionStoreEntry[]): Promise<void>;
  load(key: SessionKey): Promise<SessionStoreEntry[] | null>;

  // Optional
  listSessions?(
    projectKey: string,
  ): Promise<Array<{ sessionId: string; mtime: number }>>;
  delete?(key: SessionKey): Promise<void>;
  listSubkeys?(key: {
    projectKey: string;
    sessionId: string;
  }): Promise<string[]>;
};
```

`SessionKey` addresses one transcript. `projectKey` is a stable, filesystem-safe encoding of the working directory, `sessionId` is the session UUID, and `subpath` is set when the entry belongs to a subagent transcript or sidecar file rather than the main conversation. Treat `subpath` as an opaque key suffix; it follows the on-disk layout, for example `subagents/agent-<id>`. When `subpath` is undefined the key refers to the main transcript.

Method	Required	Called when
<code>append</code>	Yes	After each batch of transcript entries is written locally. Entries are JSON-safe objects, one per line in the local JSONL.
<code>load</code>	Yes	Once before the subprocess spawns, when <code>resume</code> is set. Return <code>null</code> if the session is unknown.
<code>listSessions</code>	No	By <code>listSessions({ sessionStore })</code> and by <code>query() / startup()</code> with <code>continue: true</code> . If undefined, those calls throw.
<code>delete</code>	No	By <code>deleteSession({ sessionStore })</code> . Deleting the main key (no <code>subpath</code>) must cascade to all subkeys for that session. If

Method	Required	Called when
		undefined, deletion is a no-op, which suits append-only backends.
<code>listSubkeys</code>	No	During resume, to discover subagent transcripts. If undefined, only the main transcript is restored.

Quick start

The SDK ships an `InMemorySessionStore` for development and testing. The example below runs a query with the store attached, captures the session ID from the result message, then resumes from the store in a second `query()` call. The second call passes the same store instance plus `resume`, so the SDK loads the transcript from the store instead of the local filesystem:

```
import { query, InMemorySessionStore } from "@anthropic-ai/claude-agent-sdk";

const store = new InMemorySessionStore();

let sessionId: string | undefined;
for await (const message of query({
  prompt: "List the TypeScript files under src/",
  options: { sessionStore: store },
})) {
  if (message.type === "result") {
    sessionId = message.session_id;
  }
}

// Resume from the store. The agent has full context from the first call.
for await (const message of query({
  prompt: "Summarize what those files do",
  options: { sessionStore: store, resume: sessionId },
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

The second query prints a summary of the files from the first query, which shows the agent resumed with full context from the store.

Write your own adapter

Implement `append` and `load` against your backend. Add `listSessions` , `delete` , and `listSubkeys` if you want `listSessions()` , `deleteSession()` , and subagent resume to work against the store.

Entries passed to `append` are typed as `SessionStoreEntry` (a `{ type: string; ... }` object). Treat them as opaque JSON-safe values: persist them in order and return them from `load` in the same order. `load` must return entries that are deep-equal to what was appended; byte-equal serialization is not required, so backends like Postgres `jsonb` that reorder object keys are fine.

Reference implementations

The TypeScript SDK repository includes runnable reference adapters for S3, Redis, and Postgres under `examples/session-stores/` . They are not published to npm; copy the `src/` file you need into your project and install the corresponding backend client.

Adapter	Backend client	Storage model
<u><code>S3SessionStore</code></u>	<code>@aws-sdk/client-s3</code>	One JSONL part file per <code>append()</code> ; <code>load()</code> lists, sorts, and concatenates.
<u><code>RedisSessionStore</code></u>	<code>ioredis</code>	<code>RPUSH</code> / <code>LRANGE</code> list per transcript, plus a sorted-set session index.
<u><code>PostgresSessionStore</code></u>	<code>pg</code>	One row per entry in a <code>jsonb</code> table, ordered by <code>BIGSERIAL</code> .

Each adapter takes a pre-configured client instance, so you control credentials, TLS, region, and pooling. For example, with S3:

```

import { query } from "@anthropic-ai/claude-agent-sdk";
import { S3Client } from "@aws-sdk/client-s3";
import { S3SessionStore } from "../S3SessionStore"; // copied from
examples/session-stores/s3

const store = new S3SessionStore({
  bucket: "my-claude-sessions",
  prefix: "transcripts",
  client: new S3Client({ region: "us-east-1" }),
});

for await (const message of query({
  prompt: "Hello!",
  options: { sessionStore: store },
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}

// Later, possibly on a different host:
for await (const message of query({
  prompt: "Continue where we left off",
  options: { sessionStore: store, resume: "previous-session-id" },
})) {
  // ...
}

```

Validate your adapter

Both SDKs ship a conformance suite that asserts the behavioral contract `append` , `load` , and the optional methods must satisfy. Tests for optional methods skip automatically when those methods are not implemented.

In TypeScript, copy `shared/conformance.ts` from the example directory into your test suite. In Python, the suite ships in the package:

Python

```
import pytest
from claude_agent_sdk.testing import run_session_store_conformance

@pytest.mark.asyncio
async def test_my_store_conformance():
    await run_session_store_conformance(MyRedisStore)
```

Behavior notes

Dual-write architecture

The store is a mirror, not a replacement. The Claude Code subprocess always writes to local disk first; the SDK then forwards each batch to `append()`. If you want the local copy to be ephemeral, point `CLAUDE_CONFIG_DIR` at a temp directory in `options.env`. Because the mirror depends on local writes, `sessionStore` cannot be combined with `persistSession: false`; the SDK throws if you set both. It also throws if combined with `enableFileCheckpointing`, since file-history backup blobs are written directly to local disk and are not mirrored to the store.

Mirror writes are best-effort

If `append()` rejects or times out, the error is logged, a `{ type: "system", subtype: "mirror_error" }` message is emitted into the iterator, and the query continues. The local transcript is already durable on disk, so a store outage does not interrupt the agent or lose data locally. Batches that fail are not retried, so monitor for `mirror_error` if you need to detect store data loss.

`getSessionMessages` returns the post-compaction chain

`getSessionMessages({ sessionStore })` returns the linked message chain the agent would see on resume. After auto-compaction, earlier turns are replaced by a summary, so a session whose store holds 503 raw entries may return 18 messages from `getSessionMessages`. For the full raw history, including pre-compaction turns and metadata entries, call `store.load(key)` directly.

`forkSession` is not a byte copy

`forkSession({ sessionStore })` reads the source entries, rewrites every `sessionId` field and

remaps message UUIDs, then appends the transformed entries under a new key. An adapter-level copy or `CopyObject` shortcut would produce a transcript that still references the old session ID, so the SDK does not use one.

Subagent transcripts

Subagent transcripts are mirrored under `subpath: "subagents/agent-<id>"`. `listSubagents({ sessionStore })` requires the adapter to implement `listSubkeys`; `getSubagentMessages({ sessionStore })` uses it when available but falls back to the direct subpath when it is undefined. Resume also calls `listSubkeys` to restore subagent files; without it, only the main transcript is materialized.

Retention

The SDK never deletes from your store on its own. Retention is the adapter's responsibility: implement TTLs, S3 lifecycle policies, or scheduled cleanup according to your compliance requirements. Local transcripts under `CLAUDE_CONFIG_DIR` are swept independently by the `cleanupPeriodDays` setting.

Supported on

The following SDK functions accept a `sessionStore` option and operate against the store instead of the local filesystem when it is provided:

- `query()`
- `startup()`
- `listSessions()`
- `getSessionInfo()`
- `getSessionMessages()`
- `renameSession()`
- `tagSession()`
- `deleteSession()`
- `forkSession()`
- `listSubagents()`
- `getSubagentMessages()`

Related resources

- [Work with sessions](#): Continue, resume, and fork without a custom store
- [Host the SDK](#): Deployment patterns for multi-host environments
- [TypeScript Options](#) : Full option reference
- [examples/session-stores/](#) : Runnable S3, Redis, and Postgres reference adapters